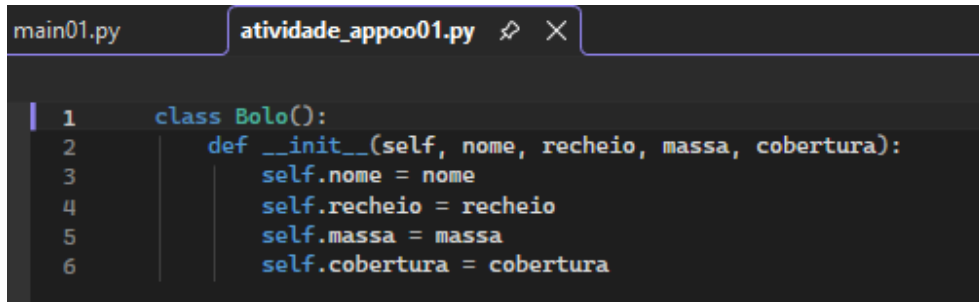


Atividade Prática 1 – parte 1 – APPOO

1 (Aula 05)

Criação da instância da classe *Bolo* e adição seus atributos:

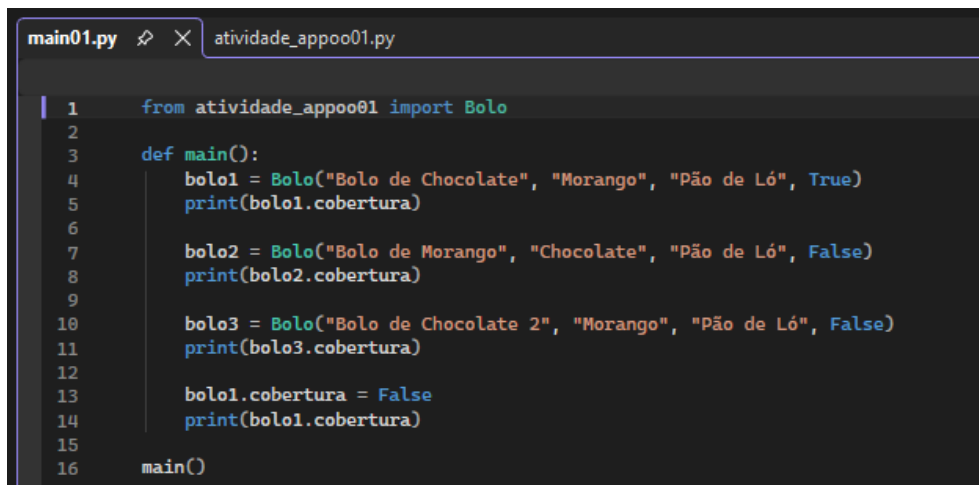
- *nome* como string;
- *recheio* como string;
- *massa* como string;
- *cobertura* como bool (apenas retorna true se tem cobertura).



```
main01.py  atividade_appoo01.py ✕ ×  
  
1  class Bolo():  
2      def __init__(self, nome, recheio, massa, cobertura):  
3          self.nome = nome  
4          self.recheio = recheio  
5          self.massa = massa  
6          self.cobertura = cobertura
```

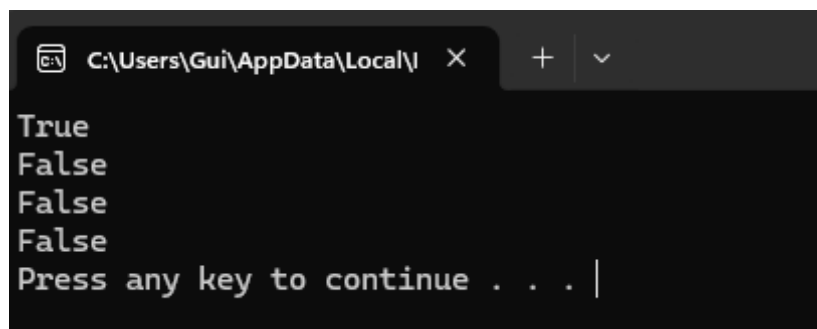
Figura 1 Módulo principal com classe Bolo

Criei os três primeiros objetos da classe Bolo (bolo1, bolo2 e bolo3) e logo depois eu alterei diretamente o valor do atributo cobertura do objeto Bolo1 agora ele passa a não ter mais cobertura — o esperado é que na quarta linha seja impresso “false”.



```
main01.py ✕ ×  atividade_appoo01.py  
  
1  from atividade_appoo01 import Bolo  
2  
3  def main():  
4      bolo1 = Bolo("Bolo de Chocolate", "Morango", "Pão de Ló", True)  
5      print(bolo1.cobertura)  
6  
7      bolo2 = Bolo("Bolo de Morango", "Chocolate", "Pão de Ló", False)  
8      print(bolo2.cobertura)  
9  
10     bolo3 = Bolo("Bolo de Chocolate 2", "Morango", "Pão de Ló", False)  
11     print(bolo3.cobertura)  
12  
13     bolo1.cobertura = False  
14     print(bolo1.cobertura)  
15  
16     main()
```

Figura 2 Main com testes e objetos criados



```
C:\Users\Gui\AppData\Local\I  ✕  +  v  
  
True  
False  
False  
False  
Press any key to continue . . . |
```

Figura 3 Saída do código no terminal

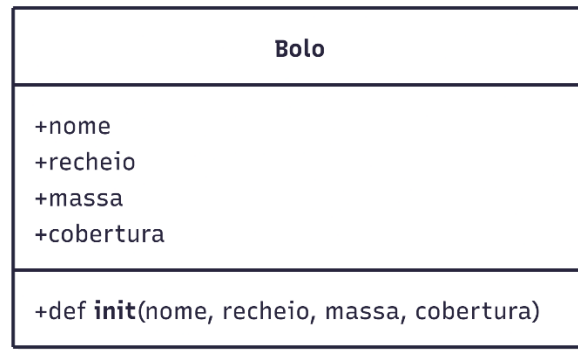


Figura 4 Diagrama UML

2 (Aula 05)

Criação da instância da classe *FazBolo* e adição seus atributos e método *assar*:

```

main02.py  atividade_apoo02.py  x
FazBolo  tempo
1  class FazBolo():
2      def __init__(self, massa, recheio, cobertura):
3          self.massa = massa
4          self.recheio = recheio
5          self.cobertura = cobertura
6
7      def assar(self, temperatura, tempo):
8          if temperatura not in ["alta", "media", "baixa"]:
9              print("Temperatura inválida. Por favor, escolha entre 'alta', 'media' ou 'baixa'.")
10             return
11          if tempo <= 0 or tempo > 180:
12              print("Tempo inválido. Por favor, insira um tempo positivo e menor ou igual a 180.")
13              return
14          else:
15              self.temperatura = temperatura
16              self.tempo = tempo
17
18          print(
19              f"Você está querendo assar o bolo de massa com recheio de {self.recheio} e cobertura de {self.cobertura} no fogo temperatura por "
20              f"{tempo} minutos", sendo {tempo} e {temperatura} inseridos pelo usuário"
21          )
22
23          if temperatura == "alta":
24              print("Para o fogo alto, o bolo assa em 10 minutos. Antes fica cru e depois ele queima.")
25
26          elif temperatura == "media":
27              print("Para o fogo médio, o bolo assa em 30 minutos. Antes fica cru e depois ele queima.")
28
29          elif temperatura == "baixa":
30              print("Para o fogo baixo, o bolo assa em 45 minutos. Antes fica cru e depois ele queima.")

```

Figura 5 Módulo principal com classe *FazBolo*, instância e método *assar*

Coloquei as condicionais necessárias para que os valores não sejam negativos ou ultrapassem 180.

Main com os testes realizados com valores absurdos e negativos:

```
main02.py  atividade_appoo02.py*
1  from atividade_appoo02 import FazBolo
2  def main():
3      Bolo1 = FazBolo("Pão de Ló", "Morango", "Chocolate")
4      Bolo1.assar("media", 90)
5
6      print("\n")
7
8      Bolo2 = FazBolo("Chocolate", "Doce de Leite", "Brigadeiro")
9      Bolo2.assar("alta", 100000)
10
11     print("\n")
12
13     Bolo3 = FazBolo("Baunilha", "Creme", "Frutas")
14     Bolo3.assar("baixa", -45)
15
16     main()
```

Figura 6 Main com testes e objetos criados

```
C:\Users\Gui\AppData\Local\  +  -  x
Você está querendo assar o bolo de massa com recheio de Morango e cobertura de Chocolate no fogo temperatura por 90 min
utos", sendo 90 e media inseridos pelo usuário
Para o fogo médio, o bolo assa em 30 minutos. Antes fica cru e depois ele queima.

Tempo inválido. Por favor, insira um tempo positivo e menor ou igual a 120.

Tempo inválido. Por favor, insira um tempo positivo e menor ou igual a 120.
Press any key to continue . . . |
```

Figura 7 Saída no terminal

Funcionou corretamente, para valores negativos e absurdos ele exibe a mensagem de tempo inválido e para valores correto, ele exibe a mensagem esperada.

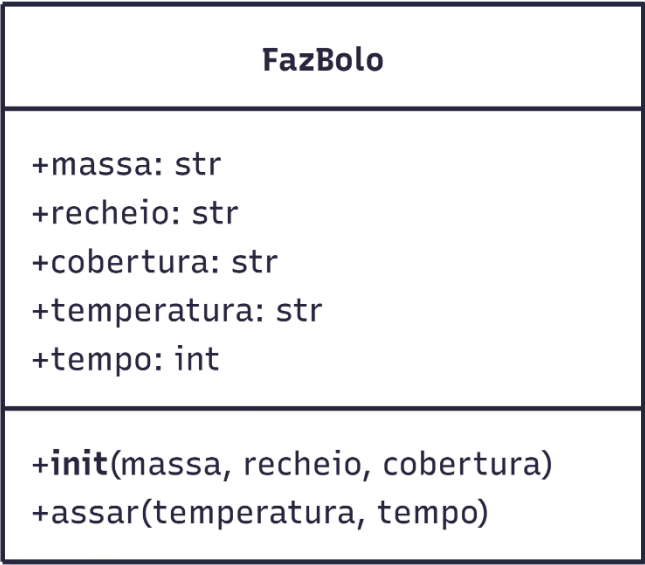


Figura 8 Diagrama UML

3 (Aula 05)

Criação da instância da classe *CarroAlugado* e *Locadora* e adição seus respectivos atributos e métodos:

```
main03.py  atividade_app03.py  ✖  ✕

determine_valor_aluguel
1  class CarroAlugado:
2      def __init__(self, modelo, quantidade_dias, quilometros_rodados):
3          if quantidade_dias < 0:
4              raise ValueError("A quantidade de dias não pode ser negativa.")
5          if quilometros_rodados < 0:
6              raise ValueError("A quantidade de quilômetros rodados não pode ser negativa.")
7          self.modelo = modelo
8          self.quantidade_dias = quantidade_dias
9          self.kilometros_rodados = quilometros_rodados
10
11     def forneca_valor_aluguel(self):
12         dia: int = 0
13         valor_diaria: float = 120.00
14         valor_km : float = 0.80
15         return self.quantidade_dias * valor_diaria + self.kilometros_rodados * valor_km
16
17     class Locadora:
18         def __init__(self, nome_locadora):
19             self.nome_locadora = nome_locadora
20
21         def determine_valor_aluguel(self, carro: CarroAlugado):
22             valor_aluguel = carro.forneca_valor_aluguel()
23             print(f"O carro {carro.modelo}, alugado por dias {carro.quantidade_dias} na locadora
24                   {self.nome_locadora} teve o valor total de {valor_aluguel} reais, considerando
25                   {carro.quantidade_dias} dias de aluguel e {carro.kilometros_rodados} km rodados.")
```

Figura 9 Módulo principal com classes, instâncias e métodos

Aqui há a troca de mensagens entre objetos, quando `valor_aluguel = carro.forneca_valor_aluguel()`, pois um método da classe *Locadora* solicitando a um método da classe *CarroAlugado*.

Para o aviso de erro de valor, usei a função `raise ValueError()`, que lança uma exceção quando um valor é negativo.

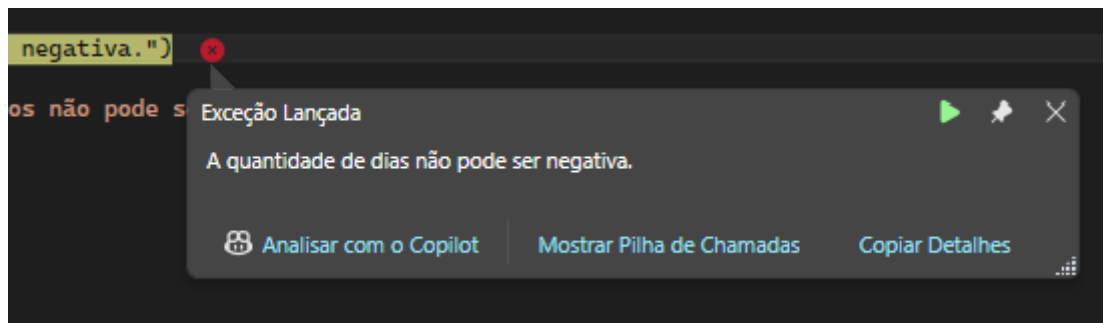


Figura 10 Exceção lançada quando dia é um valor negativo

Main com testes corretos:

```
main03.py atividade_appoo03.py
1 from atividade_appoo03 import CarroAlugado, Locadora
2
3 def main():
4     locadora1 = Locadora("Locadora A")
5     carro1 = CarroAlugado("Fusion", 5, 300)
6     locadora1.determine_valor_aluguel(carro1)
7
8     locadora2 = Locadora("Locadora B")
9     carro2 = CarroAlugado("Civic", 3, 150)
10    locadora2.determine_valor_aluguel(carro2)
11    main()
```

Figura 11 Main com objetos criados e testes feitos

Em `locadora1.determine_valor_aluguel(carro1)` e `locadora2.determine_valor_aluguel(carro2)` também há troca de mensagens entre os objetos.

```
C:\Users\Gui\AppData\Local\I + v
0 carro Fusion, alugado por dias 5 na locadora
  Locadora A teve o valor total de 840.0 reais, considerando
  5 dias de aluguel e 300 km rodados.
0 carro Civic, alugado por dias 3 na locadora
  Locadora B teve o valor total de 480.0 reais, considerando
  3 dias de aluguel e 150 km rodados.
Press any key to continue . . . |
```

Figura 12 Saída no terminal

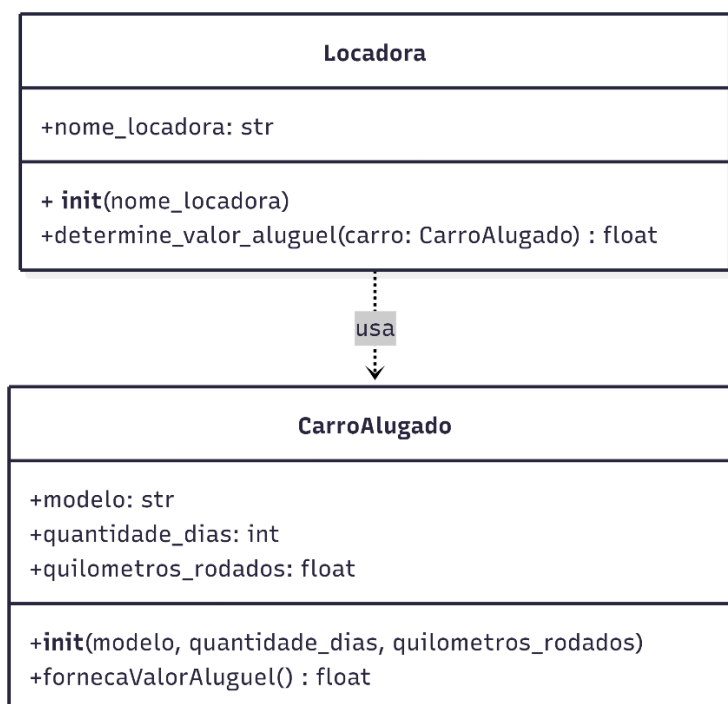
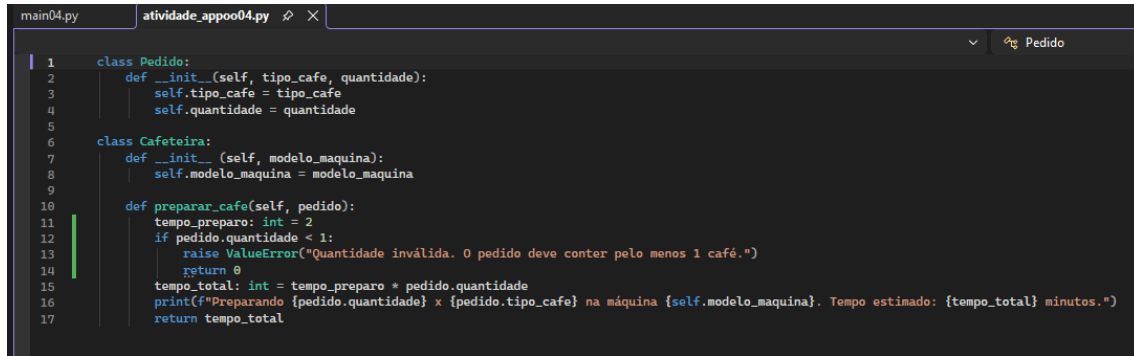


Figura 13 Diagrama UML

4 (Aula 05)

Criação da instância da classe *Pedido* e *Cafeteira* e adição seus respectivos atributos e métodos:



```
1 class Pedido:
2     def __init__(self, tipo_cafe, quantidade):
3         self.tipo_cafe = tipo_cafe
4         self.quantidade = quantidade
5
6 class Cafeteira:
7     def __init__(self, modelo_maquina):
8         self.modelo_maquina = modelo_maquina
9
10    def preparar_cafe(self, pedido):
11        tempo_preparo: int = 2
12        if pedido.quantidade < 1:
13            raise ValueError("Quantidade inválida. O pedido deve conter pelo menos 1 café.")
14        return 0
15        tempo_total: int = tempo_preparo * pedido.quantidade
16        print(f"Preparando {pedido.quantidade} x {pedido.tipo_cafe} na máquina {self.modelo_maquina}. Tempo estimado: {tempo_total} minutos.")
17        return tempo_total
```

Figura 14 Módulo principal com classes, instâncias e métodos

Novamente com a função *raise ValueError()* para lançamento de exceções.

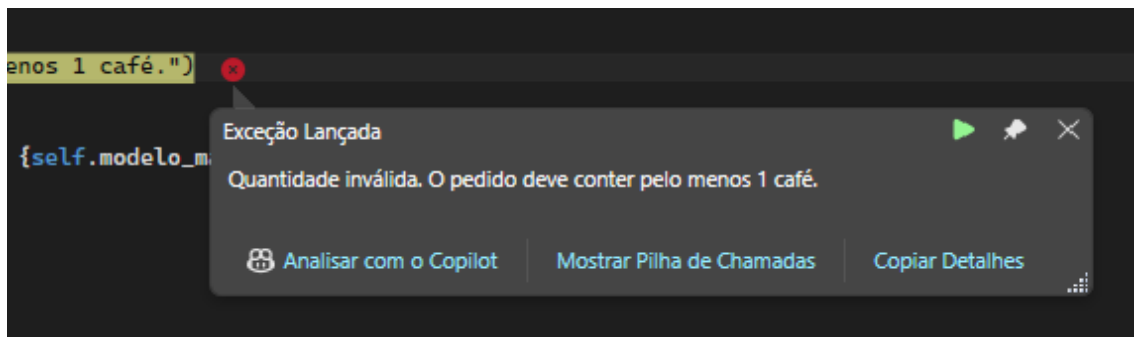
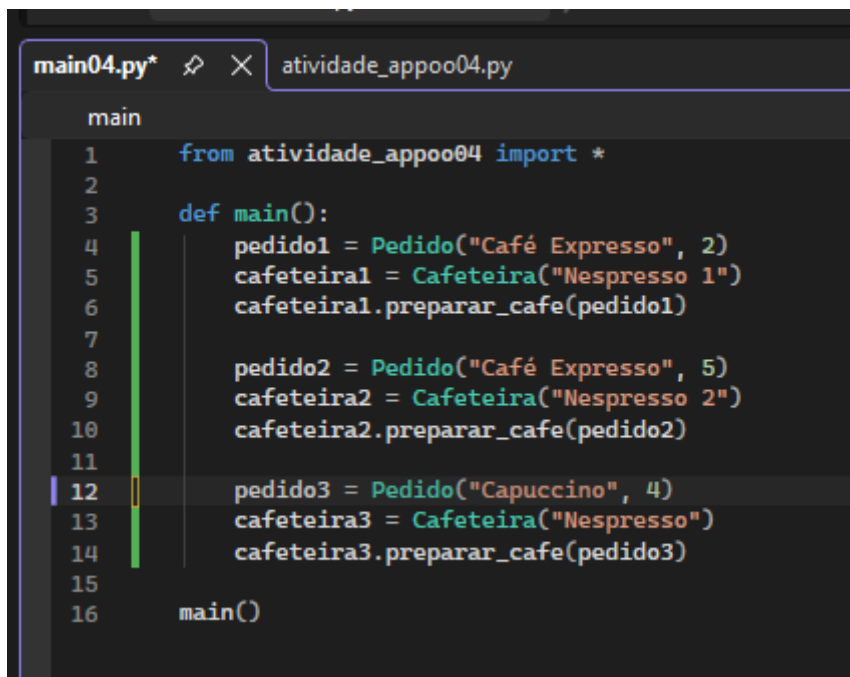


Figura 15 Exceção lançada quando pedido é um valor negativo ou 0



```
main
1 from atividade_appoo04 import *
2
3 def main():
4     pedido1 = Pedido("Café Expresso", 2)
5     cafeteira1 = Cafeteira("Nespresso 1")
6     cafeteira1.preparar_cafe(pedido1)
7
8     pedido2 = Pedido("Café Expresso", 5)
9     cafeteira2 = Cafeteira("Nespresso 2")
10    cafeteira2.preparar_cafe(pedido2)
11
12    pedido3 = Pedido("Capuccino", 4)
13    cafeteira3 = Cafeteira("Nespresso")
14    cafeteira3.preparar_cafe(pedido3)
15
16    main()
```

Figura 16 Main com objetos criados e testes feitos

Aqui há troca de mensagens entre objetos, *pedido1* é um objeto de *Pedido* e *preparar_cafe()* é um método da classe *Cafeteira*.

```
C:\Users\Gui\AppData\Local\l X + v
Preparando 2 x Café Expresso na máquina Nespresso 1. Tempo estimado: 4 minutos.
Preparando 5 x Café Expresso na máquina Nespresso 2. Tempo estimado: 10 minutos.
Preparando 4 x Capuccino na máquina Nespresso. Tempo estimado: 8 minutos.
Press any key to continue . . . |
```

Figura 17 Saída no terminal

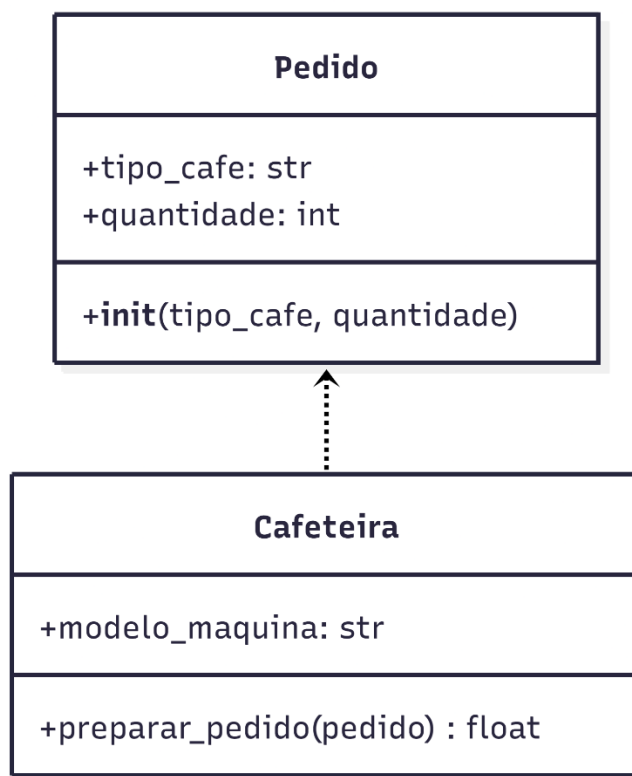


Figura 18 Diagrama UML